

```
$ python3 day01.py
```

Python Day01

0000000000

```
>>> print("00000000000000")
```

```
00000000000000
```



```
$ cat agenda.md
```



1. Python Day01 — □□□□□□□□□□

2. □□□□

3. □□□□□□□□□□

4. □□□□□□□□

5. □□□□□□□□□□□□

6. □□

7. □□□□

8. □□□□□□

9. □□□□□□□□

10. Output

11. □□□□□□□□□□□□

12. □□□□□□□

13. f-string □□□□□□



Python

NOT ENOUGH NOT ENOUGH NOT ENOUGH NOT ENOUGH NOT ENOUGH NOT ENOUGH NOT ENOUGH NOT ENOUGH NOT ENOUGH NOT ENOUGH (type) NOT ENOUGH NOT ENOUGH NOT ENOUGH NOT ENOUGH NOT ENOUGH NOT ENOUGH NOT ENOUGH NOT ENOUGH NOT ENOUGH NOT ENOUGH NOT ENOUGH NOT ENOUGH NOT ENOUGH NOT ENOUGH NOT ENOUGH NOT ENOUGH NOT ENOUGH

10

int
□□

12.3

float
□□□

'hi'

str
□□

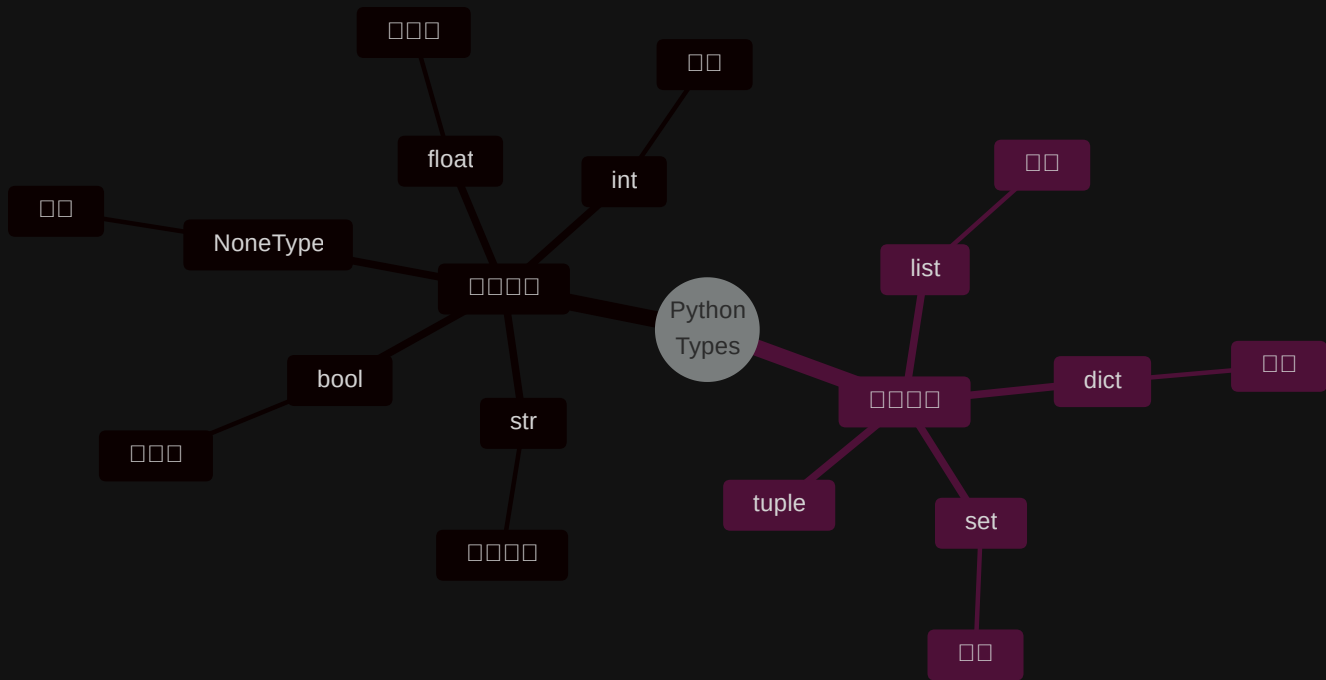
True

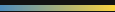
bool
□□

None

NoneType
□□

```
>>> type(10) # <class 'int'>
>>> type('hi') # <class 'str'>
```





```
# [] [] [] [] []
'123' ~ [] []
123 ~ [] []
'123' == 123 → False
```

'123'



123



```
>>> '123' == 123
False # [] [] [] [] [] [] [] []
```

🔑 [] [] [] [] [] `input()` [] [] [] []








































🤔

1. $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

2. $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

3. $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

4. $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

5. $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

6. $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

7. $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

8. $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

9. $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

10. $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

11. $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

12. $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

13. $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

14. $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

15. $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

16. $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

17. $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

18. $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

19. $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

20. $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$



□ □ □ □ □ □ □ □ □ □ □ □



□ □ □ □ □ □ □ □ □ □

if, for, while



□□□□ □□□□ □□□

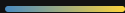
```
1    a, b = 10, 20
2    print(a)    # 10
3    print(b)    # 20
```

✨ □□□□ Python —————

PART 03

Output

```
❯ print() [] [] [] [] []
```



```
1 print('hi')
2 print('hi')      # 00000000 hi 000
```

00000

```
1 hi
2 hi
```



```
1 # 😞 000 - 000
2 name = 'Lucas'
3 age = 18
4 print('Hello,', name, 'you are', age, 'years old')
```

```
1 # 😊 0 f-string - 0000
2 name = 'Lucas'
3 age = 18
4 print(f'Hello, {name}! You are {age} years old.')
```

```
1 # 🌟 f-string 0000000
2 name = 'Lucas'
3 age = 18
4 print(f'Hello, {name}! Next year you will be {age + 1}')
```



print()



□□□□ □□□□ □□□□ □□□□

```
1 print('hi')
2 print('hi')
3 # :
4 # hi
5 # hi
```

🔑 end — '\n' · sep — ' ' □

f-string

f-string

```
1  # 
2  print(f'{}')
3
4  # 
5  pi = 3.14159265
6  print(f'π ≈ {pi:.2f}')    # π ≈ 3.14
7  print(f'π ≈ {pi:.4f}')    # π ≈ 3.1416
8
9  # 
10 print(f'{42:05d}')        # 00042
```

```
{:}
:.2f  - 
:05d  - 
```

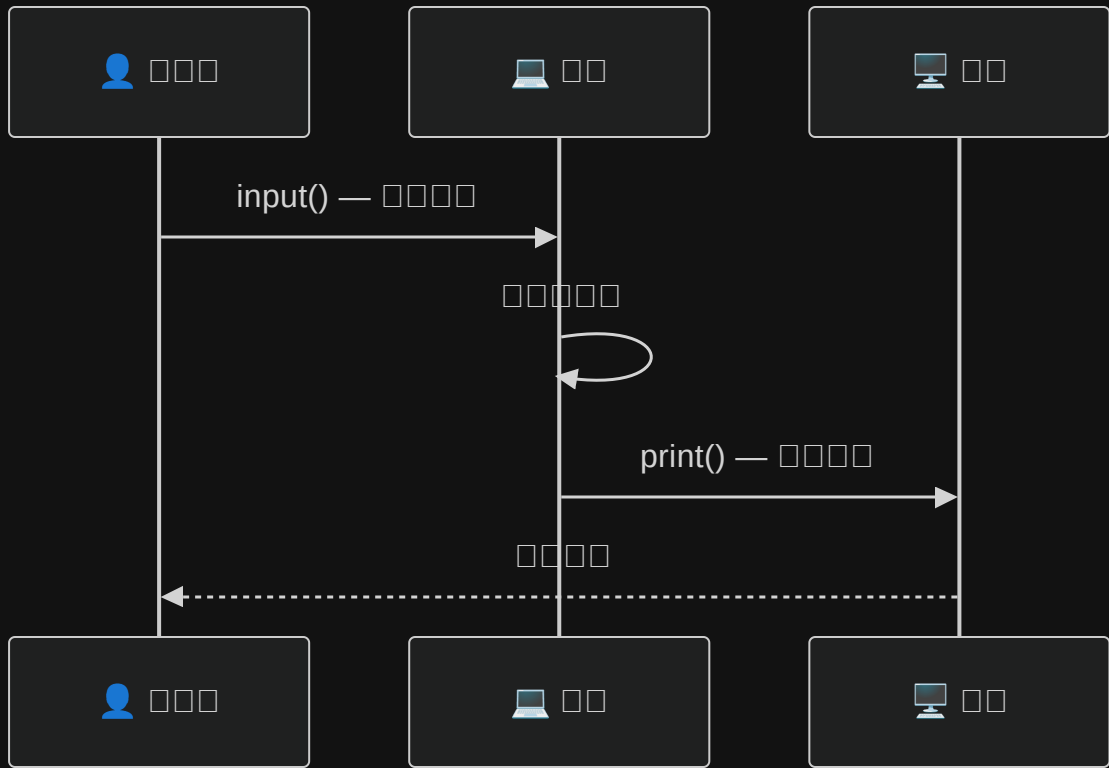


```
1  pi = 3.14159265
2  print(f'π ≈ {pi:.2f}')
3  print(f'π ≈ {pi:.4f}')
4  print(f'{42:05d}')
```

PART 04

Input

□□□□□□□□□□





`input()` □□□□□□□□□□□□

```
1 n = input()
2 nn = input('□□□□□□□□')
```

⚠ `input()` □□□□□□□□□□□□□□□□
! '123' != 123



```
1 # □□□□□□□□□□
2 name = input('□□□□□ ')
3 print(f'Hello, {name}!')
4
5 # input() □□□□□□□□
6 age_str = input('□□□ ')
7 print(type(age_str)) # <class 'str'>
8
9 age = int(age_str) # □□□□
10 print(f'□□□□ {age + 1} □□')
```



□□□□□□□□□□□□□□□□

sys.stdin while + try □□□□

```
1  import sys
2
3  for line in sys.stdin:
4      line = line.strip()
5      print(line)
```

 sys.stdin

□□□
□□□□□□

 while+try

□□□□
□□□□□□

 □□□□

□□□
□□□□□□



a.py ☐

--	--	--	--	--	--

□ □ □ □ □

in.txt ☐

```
1 Everything will get better 12345
```



□□□□□□□□

  in.txt □□□□□□□□□□□□□□□□

```
1 import sys
2
3 # UTF-8
4 sys.stdin.reconfigure(encoding='utf-8')
5
6 for line in sys.stdin:
7     line = line.strip()
8     print(line)
```

□□□□□□□□

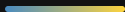
```
1 python a.py < in.txt > out.txt
```

 UTF-8 Windows□□□□□□



□□	□□□	□□	□□□□
□□	<code>n + 2</code>	Addition	<code>n=10 → 12</code>
□□	<code>n - 2</code>	Subtraction	<code>n=10 → 8</code>
□□	<code>n * 2</code>	Multiplication	<code>n=10 → 20</code>
□□	<code>n / 2</code>	Division Not Integer Not Integer Not Integer Not Integer float□	<code>n=10 → 5.0</code>
□□□□	<code>n // 2</code>	□□□□□□□□	<code>n=10 → 5</code>
□□	<code>n ** 6</code>	Power	<code>n=10 → 1000000</code>
□□□	<code>n % 6</code>	Modulo□□□□□□□□□□	<code>n=10 → 4</code>

`int``int``int``/``/``*``*``*``^``^``^``^`



```
1  n = 10
```

```
n + 2 → 12
```

```
n - 2 → 8
```

```
n * 2 → 20
```

```
n / 2 → 5.0 ← float
```

```
n // 2 → 5
```

```
n ** 6 → 1000000
```

```
n % 6 → 4 ← 10 ÷ 6 商
```

💡 / `float` // `float`

□ □ □ □ □ □ □

□ □ □ □ □ □ □ □ □ □

□ □ □ □ □ □ □ □ □ □

```
1  x = 10
2  x = x + 5    # x 15
3  x = x * 2    # x 30
4  print(x)
```

⚡ `x += 1` `10` `11` `12` `13` `14` `15` `16` `17` `18` `19` `20` `21` `22` `23` `24` `25` `26` `27` `28` `29` `30` `31` `32` `33` `34` `35` `36` `37` `38` `39` `40` `41` `42` `43` `44` `45` `46` `47` `48` `49` `50` `51` `52` `53` `54` `55` `56` `57` `58` `59` `60` `61` `62` `63` `64` `65` `66` `67` `68` `69` `70` `71` `72` `73` `74` `75` `76` `77` `78` `79` `80` `81` `82` `83` `84` `85` `86` `87` `88` `89` `90` `91` `92` `93` `94` `95` `96` `97` `98` `99` `100`

□□□□□□□□

□□□□□□□□□□

■ ✓ True □□□

■ ✗ False □□□

□□□	□□
<	□□
<=	□□□□
>	□□
>=	□□□□
==	□□
!=	□□□

□□

```
1  10 <= 60      # True
2  123 == 123    # True
3  20 == 40      # False
4  '123' == 123  # False ⚠
```

⚠ '123' 123
False

Logical Operators

Python Logical Operators

Operator	Description
<code>and</code>	Both operands must be true
<code>or</code>	At least one operand must be true
<code>not</code>	Reverses the logical state of its operand

```
1 age = 18
2 has_id = True
3
4 age >= 18 and has_id    # True
5 age < 18 or has_id     # True
6 not has_id              # False
```

🧠 `'and'` = Both operands must be true · `'or'` = At least one operand must be true · `'not'` = Reverses the logical state of its operand

Truthy / Falsy □□□□□□

Python True / False □□□□□□

```
1 bool(0)           # False
2 bool(1)           # True
3 bool("")           # False
4 bool("hi")         # True
5 bool(None)         # False
6 bool([])           # False
```

Falsy False

- 0
- " "
- None
- []
- {}

Truthy True

- 0
-
-

if □□□□

□□□□□□□□

```
1 age = 18
2
3 if age >= 18:
4     print("□□□□")
```

⚠ Python □□□□□□□□□□□□□□□□

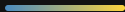
if / elif / else

□□□□□□

```
1  score = 85
2
3  if score >= 90:
4      print("A")
5  elif score >= 80:
6      print("B")
7  elif score >= 70:
8      print("C")
9  else:
10     print("F")
```



if score >= 90: print("A")
elif score >= 80: print("B")
elif score >= 70: print("C")
else: print("F")



```
1 username = input("000")
2 password = input("000")
3
4 if username == "admin" and password == "1234":
5     print("0000")
6 else:
7     print("0000")
```

🔑 000000input + 00 + and 0000

[illegible]



- Python □□□□□int / float / str / bool / None

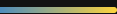
- `input()` □□□□□□□□

- `print()` □□□□□ f-string □□□

- `/` □ `//` □ `%` □□□□□

- □□□□□□□□ + □□□□□□□□

- `if` / `elif` / `else` □□□□



- ☐ ☐ ☐ ☐ ☐
☐ ☐ ☐ ☐
☐ ☐ ☐ ☐ ☐ ☐
☐ ☐ ☐ ☐ ☐



MDH&D

NO GLYPH
11-03-2008

WILEY

11

Loop